Bipartite Graph and Suggestion Algorithms in C

INF224 Data Structures and Algorithms, Term Project Report

Hasan Alper Ayaz

28 December 2024 (English edition, October 2026)

Abstract

This report describes how a bipartite graph is modelled in the C programming language and the recommendation algorithms built on that model. It explains which data structure holds the graph and how the data is loaded into it. Five recommendation algorithms are described step by step, their time and space complexity is evaluated, and the other helper functions of the project are summarised.

The program recommends movies to a user of the MovieLens 100K data set (943 users, 1,682 movies, 100,000 ratings). Users and movies are the two sets of nodes, and a rating is an edge between a user and a movie whose weight is the rating (1 to 5).

The code described here is the version in this repository. It was written in December 2024 and corrected in October 2026; the section *Changes since December 2024* at the end lists what changed and how the corrections were checked.

1. Storing the data

Every rated movie is kept in a node of the following structure:

```c
typedef struct MovieNode {
    int movieID;
    int rating;
    int degree;      // how many users rated the movie
    int distance;    // weighted distance, used by the shortest-path method
    struct MovieNode *next;
} MovieNode;
```

Each user is represented by the structure below, which holds the head of the linked list of the movies the user rated:

```c
typedef struct User {
    int userID;
    MovieNode *movies;
} User;
```

The graph itself is an array of `User` structures:

```
User users[MAX_USERS];

users[0] -> (movie 12, rating 4) -> (movie 7, rating 3) -> (movie 1, rating 5) -> NULL
users[1] -> (movie  3, rating 5) -> (movie 7, rating 2) -> NULL
users[2] -> (movie 12, rating 1) -> NULL
   ...
users[MAX_USERS - 1] -> ... -> NULL
```

The movie IDs and the number of movies in each list are independent for every user. A rating is an edge: it is stored once, in the list of the user who gave it. Whenever the algorithms need the other direction (all users who rated a movie), they scan the lists of all users.

2. Initialising and filling the graph

`initializeUsers()` sets the `userID` of each element of `users[]` to its index and its `movies` pointer to `NULL`. The array has `MAX_USERS` elements. Users are therefore stored at their own number minus one: user 1 is `users[0]`. Every function that takes a user number from the menu subtracts one before it uses the array.

`addMovie(userID, movieID, rating)` allocates a new `MovieNode`, stores the movie ID and the rating in it, and inserts it at the head of the user's list: the new node's `next` points to the previous head, and the user's `movies` pointer then points to the new node. Insertion is O(1). The node is allocated with `calloc`, so `degree` and `distance` start at zero, and a failed allocation ends the program with a message.

`loadInteractions(filename)` reads the data file line by line. The first three integers on a line are taken as user ID, movie ID and rating; any further column (the timestamp of the original MovieLens file) is ignored. Lines that contain no three integers, such as comments, are ignored. A line is skipped, with one warning at the end, if the user is outside 1 to `MAX_USERS`, the movie is outside 1 to `MAX_MOVIES`, the rating is outside 1 to 5, or the same user-movie pair has already been read. A file without a single valid line is an error. The path of the file is the first command-line argument; without an argument the program reads `interactions.txt` in the current directory. At the end of the program `freeUsers()` releases all lists.

3. The recommendation array

Every algorithm delivers its result in the global integer array `recommendations[]`, whose size is the macro `MAX_RECOMMEND` (5). The array starts with zeros, but each algorithm needs the value `-1` to mean "no recommendation". `resetRecommendations()` fills the array with `-1`, and `doRecommend()` calls it at the beginning of every request. This matters: results left behind by an earlier algorithm would otherwise be shown as if the new algorithm had produced them. `printRecommendations()` prints the elements up to the first `-1`; if fewer than five movies were found it adds a note saying so.

4. The recommendation algorithms

4.1 Random recommendation (`recommendRandom`)

The function takes the (zero-based) index of the user. It resets the array and then draws random movie numbers from 1 to `MAX_MOVIES` with `rand()`, which is seeded once at the start of the program

with `time.h`. A movie is added to `recommendations[]` if the user has not rated it (checked with `hasWatched()`) and it has not been drawn before in the same request, so the five movies are distinct. The number of draws is limited to 100,000, so a user who has seen almost every movie cannot cause an endless loop.

4.2 Most linked movies (`calculateDegrees`, `recommendMostLinked`)

Two functions work together. `calculateDegrees()` uses the global array `MovieNode movies[]`, indexed by movie ID. It first sets the `degree` of every element to 0 and its `movieID` to -1, then walks the lists of all users and, for every rating found, increases the `degree` of that movie by one and stores its ID. When the walk ends, the `degree` of a movie is the number of ratings it received.

`recommendMostLinked()` resets the array and sorts `movies[]` by degree, from largest to smallest, with `sort()`. The most linked movies are now at the start. Starting from the beginning, the movies the user has not rated are added to `recommendations[]` until five are found.

4.3 Highest rated movies of the user with the most movies in common (`findMostCommonUser`, `recommendFromCommonUser`)

`findMostCommonUser()` returns the number of the other user who has the most movies in common with the given user. For every other user it walks the movies of the given user and uses `hasWatched()` to check whether that other user rated them too; each hit increases a counter that starts at 0. The index with the largest counter is kept while the loop runs, and the first user with the maximum wins a tie. If the given user shares no movie with anybody the function returns -1.

`recommendFromCommonUser()` calls it, and, if a user was found, copies that user's movies into a `MovieNode` array with `fillUserMovies()`. The array is sorted by rating from largest to smallest with `sort()`, so the movies this user liked most come first. The movies the target user has not rated are added to `recommendations[]` until five are found. When nobody shares a movie with the target user, the menu says that there is nothing to recommend.

4.4 Movies closest in weighted distance (`findClosestMovies`)

The weighted length of a path is the sum of the weights, the ratings, of the edges along it, and the weighted distance between two nodes is the smallest such length. Because the graph is bipartite, a path alternates between users and movies. `findClosestMovies()` applies Dijkstra's algorithm from the given user.

It creates four arrays: the distance from the given user to every user, the distance to every movie, and one array each marking the users and the movies that have already been visited. All distances start at `INT_MAX` (not reached), except the distance of the given user to themselves, which is 0.

The loop repeats until no reachable node is left. In each round `getMin()` finds the unvisited user and the unvisited movie with the smallest distance, and the closer of the two becomes the current node.

- If the current node is a user, it is marked as visited. For each movie the user rated, the distance of the user plus the rating is the length of a path to that movie; if it is smaller than the distance stored so far, the movie's distance is updated.

- If the current node is a movie, it is marked as visited. All users who have not been visited yet are checked with `getRating()`; for each user who rated the movie, the distance of the movie plus that rating is a path to the user and replaces the stored distance if it is smaller.

When the loop ends, the distance to every reachable movie is exact. A `MovieNode` array is filled with all movies and their distances and degrees, and sorted twice with the stable `sort()`: by degree from largest to smallest, then by distance from smallest to largest. Movies at the same distance therefore stay ordered by degree, and then by movie ID. Ratings are small whole numbers, so many movies share a distance and this tie rule decides many recommendations. Finally the movies the user has not rated are added to `recommendations[]` from the start of the sorted array.

4.5 Highest rated movies of the user with the most similar ratings (`calculateSimilarity`, `recommendFromSimilar`)

`calculateSimilarity(userA, userB)` takes two zero-based user indices. A pointer walks over the movies rated by `userA`; for each of them the movies of `userB` are scanned until the same movie is found, so that the movies rated by both users are found. For each movie in common the absolute difference of the two ratings is added to `diffSum` (with `abs()` from `stdlib.h`) and a counter is increased.

If the users have fewer than `MIN_COMMON` movies in common, the function returns `INFINITY`; otherwise it returns `diffSum` divided by the number of movies in common, the average difference of opinion per movie. `MIN_COMMON` is 20: in the data set every user rated at least 20 movies, so a comparison based on fewer movies would say little.

`recommendFromSimilar()` computes this value between the given user and every other user and keeps the user with the smallest value, the user whose ratings are most similar. That user's movies are copied into a `MovieNode` array, sorted by rating from largest to smallest, and the movies the target user has not rated are added to `recommendations[]` until five are found. On MovieLens 100K, 35 of the 943 users have no other user with at least 20 movies in common; for them the menu says that there is nothing to recommend.

4.6 Random walk (`goToRandom`)

As an additional method, `goToRandom()` starts at a user and walks along rating edges for a number of steps (`STEPS`, 11 by default). It alternately picks a random movie the current user rated and a random user who rated that movie, printing every step with its weight and the total weighted distance. An odd number of steps ends on a movie, an even number on a user. The walk is not part of the menu; it is enabled by uncommenting the call in `main()`, as described in the README.

5. Time and space complexity

Let U be the number of users, M the number of movies, E the number of ratings (at most $U \cdot M$), k the average number of movies per user, and n the number of recommendations (`MAX_RECOMMEND`).

Algorithm	Time	Space
1. Random	expected $O(n \cdot (k + n))$	$O(1)$
2. Most linked	$O(E + M^2 + M \cdot k)$	$O(M)$
3. Most movies in common	$O(U \cdot k^2 + M^2 + M \cdot k)$	$O(M)$

Algorithm	Time	Space
4. Weighted distance	$O((U + M)^2 + M \cdot E + M^2)$	$O(U + M)$
5. Most similar ratings	$O(U \cdot k^2 + M^2 + M \cdot k)$	$O(M)$

Notes:

1. **Random.** Each draw costs O(k) for `hasWatched()` and O(n) for the distinctness check. Because most movies are unseen, the expected number of draws is close to n.
2. **Most linked.** Counting the degrees walks every list once: O(E). Bubble sort of the M movies costs $O(M^2)$, and picking the movies the user has not rated costs at most $O(M \cdot k)$.
3. **Most movies in common.** For each of the U users, each of the k movies of the target user is looked up in a list of length k: $O(U \cdot k^2)$. Sorting the chosen user's movies by bubble sort costs $O(M^2)$.
4. **Weighted distance.** Choosing the next node by a linear search costs O(U + M) and happens at most U + M times. Reaching the movies of all visited users costs O(E) in total. For each of the M movies, every unvisited user's list is searched for the movie, O(E) per movie, so $O(M \cdot E)$ overall, which dominates: about 170 million list steps on MovieLens 100K, a fraction of a second. The two bubble sorts cost $O(M^2)$. The arrays for the distances and the visited flags need O(U + M) space.
5. **Most similar ratings.** Comparing two users takes $O(k^2)$, so $O(U \cdot k^2)$ for all users, plus the sort.

6. Other functions

- `hasWatched(user, movie)` checks whether a user has rated a movie by scanning the user's list.
- `getRating(user, movie)` returns the user's rating of a movie, or `INT_MIN` if there is none.
- `sort(array, mode)` is a stable bubble sort of a `MovieNode` array over all `MOVIE_SLOTS` (`MAX_MOVIES + 1`) elements. The mode chooses the key and the direction: degree, rating, movie ID or distance, ascending or descending.
- `fillUserMovies(array, user)` fills a `MovieNode` array with the movies of a user and marks the unused elements with -1.
- `getMin(distances, size, visited)` returns the index of the unvisited node with the smallest reached distance, or -1.
- `doRecommend(user, mode)` resets the recommendation array and runs the algorithm that belongs to the chosen menu option.
- `readInt()`, `getUserID()` and `getMode()` read the menu input. Input is read one whole line at a time. Anything that is not a whole number in the allowed range is rejected with a message and asked for again, and the end of the input quits the program.
- `printChoices()` prints the menu options and `callMenu()` runs the menu.

7. Changes since December 2024

The original report and the code of December 2024 described a different method 4: a graph traversal that stopped after a fixed number of iterations and always moved on to the movie the current user had rated lowest, not to the nearest unvisited node. The code in this repository was corrected in

October 2026, and the descriptions above refer to the corrected code. The changes are, in the order of the commits:

1. **Method 4, memory errors.** The movie ID -1 (an unused slot) was used as an index (`visited[-1]`) on every run, and in about 4% of the users (36 of 943) a dead end of the traversal led to `visitedUsers[-1]` being written or `userMovies[-1]` being read. Ten users received no recommendation at all.
2. **Input.** `scanf` was replaced by line-based reading. Before, a letter at the user prompt, or the end of the input, caused an endless loop.
3. **Stale results and the random method.** The recommendation array is reset for every request, so a method that finds nothing no longer shows the results of the previous one (this affected 35 users in method 5). The random generator is seeded once, instead of on every call, and the random movies are distinct.
4. **Data loading.** Invalid IDs and ratings and repeated pairs are skipped, allocations are checked, the file path can be given on the command line, and the memory is freed at exit.
5. **Small corrections.** The sort, the degree reset and the loops over the arrays now include movie 1682, and the code compiles without warnings.
6. **Random walk.** The walk took one step too many in some runs and read a user outside the array.
7. **Method 4, Dijkstra.** The traversal was replaced by the complete Dijkstra search of Section 4.4, with the tie rule described there.

How the corrections were checked. All 943 users were run through all five methods with AddressSanitizer and UndefinedBehaviorSanitizer; the final version reports nothing. The results of methods 2, 3 and 5 were compared with independent calculations for a sample of 136 users, and the results of method 4 with an independent implementation of Dijkstra's algorithm for all 943 users. Apart from the switch to Dijkstra, the corrections left normal results alone: the results of methods 2 and 3 are identical to those of the December 2024 code for all 943 users, those of method 5 differ only for the 35 users it could not serve, and those of method 4 only for the ten users who had received nothing. The switch to Dijkstra then changed method 4 on purpose: the distances are exact, and equal distances are no longer resolved in favour of the lowest movie IDs (the average recommended movie ID rose from 24 to 208).

Limits that remain.

- The weight of an edge is the rating, so a path through low ratings is short. A movie that is close in this sense is not necessarily one the user will enjoy.
- Ratings are small whole numbers, so many movies share the same distance. For 97% of the users the fifth nearest movie is only 3 or 4 away, and ties are ordered by degree. Users with similar histories therefore get similar lists: over all 943 users, 122 different movies are recommended by method 4.
- The program does not measure the accuracy of the recommendations; this report does not claim that one method predicts taste better than another.
- The numbers of users and movies are fixed to MovieLens 100K, and linked lists with bubble sort are slow for much larger data.

Data and citation

The data set is not part of the repository: the terms of GroupLens do not allow redistribution without separate permission. It can be downloaded from https://grouplens.org/datasets/movielens/100k/; `scripts/download_data.sh` does this.

F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4, Article 19 (December 2015), 19 pages. https://doi.org/10.1145/2827872